

Music Player App

Aplicación web de música construida con **React + Vite**, con una interfaz tipo dashboard y estética pixel art. Permite explorar canciones, artistas, álbumes y playlists, buscar en tiempo real y reproducir pistas con un reproductor persistente estilo tocadiscos.

Vista general

El proyecto simula una plataforma de exploración musical con un diseño modular, responsivo y orientado a distintos tamaños de pantalla. El estado global se gestiona con Redux Toolkit, la música se consume de la API de Deezer a través de una capa de API propia con adaptadores, y la reproducción de audio se centraliza en un único gestor (**AudioManager**) compartido por todos los componentes.

Características

- **Buscador global** — búsqueda de canciones y artistas bajo demanda, al enviar el formulario.
- **Integración con Deezer** — chart, búsqueda y detalle de artistas, álbumes y playlists.
- **Capa de API con adaptadores** — los datos de Deezer se normalizan a un formato interno estable para desacoplar la UI del proveedor externo.
- **Reproductor persistente** — arquitectura **NowPlaying** + **MiniPlayer** sobre un único **AudioManager**, con controles, progreso, aguja, vinilo y estado de reproducción.
- **Gestión de estado global** — slices de **music**, **search**, **player** y **app** (tema, carga y errores), con hooks tipados para **dispatch** y **selectors**.
- **Tema claro/oscuro** — conmutable desde el header mediante **ThemeToggle**.
- **Layout persistente** — header, sidebar, footer y reproductor fuera del flujo de la página.
- **Sidebar de navegación** — secciones construidas a partir de datos declarativos.
- **Secciones de contenido** — hero, playlists destacadas, artistas y canciones en tendencia.
- **Estados de carga y vacío** — skeletons, loaders y **EmptyState** reutilizables.
- **CSS Modules** — estilos encapsulados por componente, con tokens y estilos globales.
- **Responsive** — adaptado a móvil, tablet y escritorio.

PROF

Tecnologías utilizadas

- **React 19**
- **Vite 8**
- **Redux Toolkit + React Redux** — estado global
- **React Router DOM 7** — navegación
- **React Hook Form** — formularios
- **Axios** — cliente HTTP
- **React Icons** — iconografía
- **clsx** — composición condicional de clases
- **CSS Modules**
- **JavaScript (ES6+)**
- **ESLint** — linting

Estructura del proyecto

```
client/
├── public/
├── src/
│   ├── api/                                # Capa de acceso a Deezer
│   │   ├── adapters/                      # Normalización de la respuesta
│   │   ├── client/                        # Instancia de Axios
│   │   ├── endpoints/                    # Rutas de la API
│   │   └── musicApi.js                    # Servicios de música
│   ├── assets/                            # Imágenes, iconos y animaciones
│   │   ├── animations/
│   │   ├── icons/
│   │   └── pink/
│   ├── audio/                             # AudioManager: motor de reproducción
│   ├── components/
│   │   └── common/                        # Button, Card, Loader, Skeleton,
│   │   SearchInput...
│   ├── layout/                            # Header, Sidebar, Footer
│   │   ├── Header/
│   │   │   ├── components/                # SearchBar, ThemeToggle, UserMenu
│   │   │   └── Header.jsx
│   │   └── Sidebar/
│   │       ├── components/                # Navigation, NavItem, SidebarLogo
│   │       └── data/
│   ├── player/
│   │   └── MiniPlayer/                    # Reproductor compacto con vinyl y
│   │   aguja
│   │       ├── NowPlaying/                # Vista ampliada de reproducción
│   │       └── sections/                  # HeroSection, FeaturedPlaylists,
│   │                                           # TopArtists, TrendingTracks
│   ├── hooks/                             # useAudio
│   ├── layouts/                           # AppLayout
│   ├── pages/                             # HomePage
│   ├── providers/                         # AppProviders, AudioProvider
│   ├── redux/
│   │   ├── music/                         # Chart y contenido destacado
│   │   ├── player/                       # Pista actual, cola y reproducción
│   │   ├── search/                       # Búsqueda y resultados
│   │   ├── slices/                       # Estado de app (tema, loading, error)
│   │   ├── hooks.js                       # useAppDispatch / useAppSelector
│   │   └── store.js
│   ├── routes/                            # AppRouter
│   ├── services/                          # searchService
│   ├── styles/                            # Estilos globales y tokens
│   ├── utils/                             # formatDuration
│   ├── App.jsx
│   └── main.jsx
├── eslint.config.js
└── index.html
```

```
|— package.json
|— vite.config.js
```

Arquitectura

Estado global

El store se organiza por dominio, y cada módulo sigue la misma estructura: estado inicial, selectores, thunks y slice.

Dominio	Responsabilidad
music	Chart, artistas y playlists destacados
search	Consulta, resultados, carga y error
player	Pista actual, cola y estado de reproducción
app	Tema, indicador de carga y errores globales

Reproducción de audio

La reproducción no vive en los componentes, sino en `src/audio/AudioManager.js`, un gestor único al que se accede mediante `AudioProvider` y el hook `useAudio`. Esto permite que el `MiniPlayer` y `NowPlaying` compartan el mismo `<audio>` y el mismo estado, sin duplicar la lógica de reproducción.

Integración con la API

`axiosClient` usa `baseURL: "/api"` y en desarrollo Vite actúa como proxy hacia `https://api.deezer.com` (ver `vite.config.js`). Los adaptadores de `src/api/adapters` convierten la respuesta cruda a un formato interno, de modo que cambiar de proveedor no afecte a los componentes.

Alcance actual de la búsqueda: la búsqueda cubre canciones y artistas. Deezer solo admite `artist` como filtro de campo en `/search`; `album` y `playlist` no existen, por lo que el álbum de cada pista se toma del resultado del track. Los resultados de álbumes y playlists están contemplados en el estado (`results.albums`, `results.playlists`) pero aún no se alimentan desde la API.

Instalación

1. Clonar el repositorio

```
git clone https://github.com/AnaLauDB/Music-Player-App.git
cd Music-Player-App/client
```

2. Instalar dependencias

```
npm install
```

3. Ejecutar el proyecto en desarrollo

```
npm run dev
```

La aplicación queda disponible en <http://localhost:5173>.

No se requieren variables de entorno: el proxy de Vite resuelve las llamadas a la API de Deezer durante el desarrollo. Para producción hay que habilitar la reescritura de `/api` en el servidor que aloje el build.

Scripts disponibles

Script	Descripción
<code>npm run dev</code>	Inicia el servidor de desarrollo de Vite
<code>npm run build</code>	Genera el build optimizado para producción
<code>npm run preview</code>	Previsualiza el build de producción localmente
<code>npm run lint</code>	Ejecuta ESLint sobre el proyecto

Diseño responsivo

La interfaz se adapta a diferentes resoluciones mediante:

- contenedores fluidos y rejillas flexibles
- media queries en puntos de ruptura clave
- reordenamiento de elementos en pantallas pequeñas
- escalado proporcional de espaciados y tipografías
- sidebar y navegación adaptada a cada tamaño de pantalla

Objetivo del proyecto

Crear una experiencia visual atractiva y funcional para explorar música, con una identidad pixel art consistente, una arquitectura limpia y escalable, y una experiencia de usuario fluida en cualquier dispositivo.

Autor

[AnaLauDB](#)